

Midterm Assignment

Creating Images with Diffusion Models

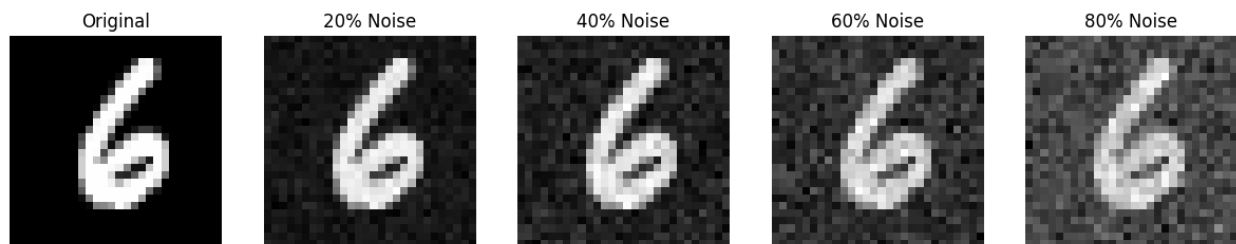
Document Contents

Understanding Diffusion	2
The Forward Diffusion Process.....	2
Gradual Noise	2
Denoise Recognition	3
Model Architecture	4
U-Net Architecture	4
Skip Connections	4
Model Conditioning	5
Training Analysis	6
Loss Value.....	6
Image Quality Evolution	6
Time Embedding Necessity.....	7
CLIP Evaluation	8
CLIP Scores Interpretation	8
Image Hypothesis.....	10
CLIP Diffusion Improvement	11
Practical Applications	12
Real World Applications	12
Model Limitations.....	12
Specific Model Improvements.....	13
Bonus Challenge.....	14
U-Net Architecture Modification	14

Understanding Diffusion

The Forward Diffusion Process

The forward diffusion process is the process of gradually "destroying" a clear image by repeatedly adding small amounts of random noise. It's like slowly adding fog to a picture step-by-step until the original image is completely hidden and only the "fog" (noise) remains.



The visualization example from the "show_noise_progression" function shows this perfectly.

- The "Original" image is a clear handwritten "6".
- At "20% Noise," the "6" is slightly fuzzy.
- By "80% Noise," the "6" is almost completely gone, replaced by what looks like random static. This shows the step-by-step degradation of the original image into pure noise.

Gradual Noise

We add noise gradually so the model can learn the reverse process: how to remove just a little bit of noise at any given step. During training, the model is given a noisy image from a random timestep and is asked to predict only the small amount of noise that was added at that specific step. If we added all the noise at once (going from a clear image straight to pure noise), the model would have the much harder task of learning to reconstruct the entire image from pure noise in a single step, but by learning to reverse the process gradually, the model can successfully "denoise" an image from pure static all the way back to a clear image one step at a time.

Denoise Recognition

In the step-by-step generation visualizations, the denoising process starts at "Step 1000" (pure noise) and ends at "Step 0" (the final image).

- For the digit '0': The image is pure noise until Step 374. At Step 374, a very faint, ghostly outline of a "0" begins to appear. It becomes clearly recognizable (though still very blurry) by Step 55.
- For the digit '3': Similarly, the digit is not visible until Step 249, where a blurry "3" shape emerges from the noise.
- For the digit '7': The "7" also becomes recognizable around Step 374.

This means we can first recognize the image approximately 60% of the way through the denoising process. Based on the three examples provided in the notebook, this point of first recognition appears to be consistent across these different digits.



Model Architecture

U-Net Architecture

The U-Net architecture is ideal for this diffusion model because its goal is an image-to-image task. The model must take a noisy image as input and output a predicted noise map of the exact same 28x28 dimensions.

A U-Net is an "encoder-decoder".

1. Downsampling Path (Encoder): It compresses the image, capturing what the features are (e.g., "this looks like a curve").
2. Upsampling Path (Decoder): It reconstructs the image back to its original size, figuring out where the features go.

This structure is perfectly suited for predicting noise, which requires understanding the image at different scales and then building a pixel-perfect map of the noise. A simpler architecture, like a standard classifier, would only be able to output a single value (like a class label), not a full-sized image.

Skip Connections

Skip connections are links that pass information directly from the U-Net's downsampling path (encoder) to its upsampling path (decoder), bypassing the central "bottleneck".

In our model's UNet forward:

1. As the image goes down each DownBlock, the output (a feature map) is saved in a list called `skip_connections`.
2. As the image is rebuilt by each UpBlock, it retrieves the corresponding feature map from that list.
3. The UpBlock then concatenates this "skip" tensor with its own upsampled tensor.

This is vital because the downsampling path captures fine-grained, high-resolution details (like sharp edges) that are lost in the compressed bottleneck. The skip connections feed this detailed information directly to the reconstruction layers, allowing the model to generate a much clearer and more precise final image (or, in this case, a more accurate noise prediction).

Model Conditioning

The model is conditioned (or "guided") to generate a specific digit by injecting information about the class label (e.g., the number '7') into the U-Net during the denoising.

This mechanism, detailed in the `train_step` and `UNet.forward` functions, works like this:

1. **One-Hot Encoding:** First, the simple class label (e.g., $c=7$) is converted into a 10-dimension one-hot vector (e.g., $[0,0,0,0,0,0,0,1,0,0]$).
2. **Embedding:** This one-hot vector is passed through an `EmbedBlock`, which is a small neural network (`nn.Linear` and `nn.GELU`). This network transforms the simple vector into a rich, high-dimensional tensor (`c_embed`) that represents the "meaning" of that class.
3. **Masking:** A "conditioning mask" (`c_mask`) is also applied, which is used for a technique called classifier-free guidance that controls how much the model should listen to the class label.
4. **Injection:** Finally, this `c_embed` tensor is added directly to the image's feature map in the middle (bottleneck) of the U-Net. This injects the "what to draw" information into the model, guiding the denoising process to create the specific digit provided.

Training Analysis

Loss Value

The loss value represents how "wrong" the model's prediction was in a given training step. As defined in the `train_step` function, the loss is the Mean Squared Error (MSE) between:

1. The actual random noise (noise) that was added to the image.
2. The predicted noise (predicted_noise) that the U-Net model generated.

A high loss value means the model's prediction was very different from the actual noise. A low loss value means the model is getting very good at accurately predicting the noise. The "Training and Validation Loss" plot shows this value decreasing over epochs, indicating that the model is learning.

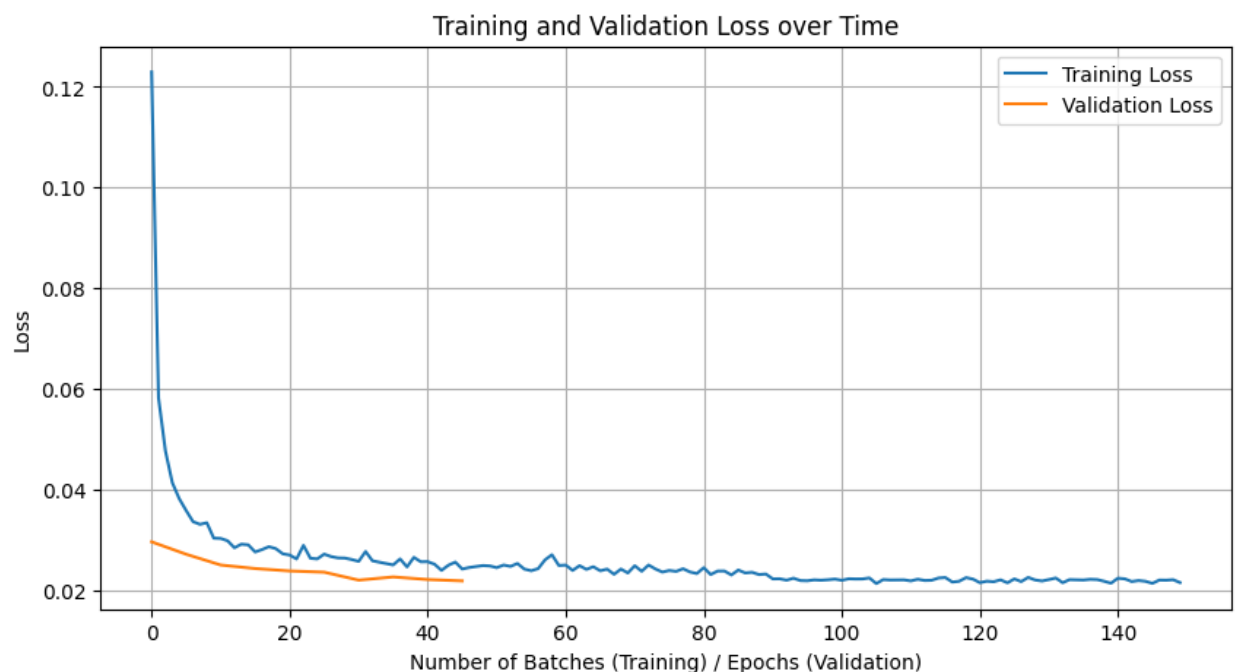


Image Quality Evolution

Based only on the training log provided in the notebook, it is impossible to know how the image quality changed.

The training loop was configured to generate sample images every 500 steps. However, the training log output shows that the training process failed during the first epoch (Epoch 1) due to a `RuntimeError: The size of tensor a (10) must match the size of tensor b (64)....` This

Benjamin LaCount II

ITAI 2376 – Fall 2025

November 1, 2025

error occurred before the first 500 steps were reached, so no samples were generated during the training run shown in the log.

The high-quality images shown later in "Step 7" must have been generated after this bug was fixed and the model was successfully trained, but the notebook does not contain the log of that successful training run.

Time Embedding Necessity

The time embedding is necessary because the model is trained to predict noise on images that are at random levels of "noisiness" (i.e., at random timesteps t).

The model needs to know how much noise it's supposed to be looking for. An image at $t=90$ (mostly static) requires a different prediction approach than an image at $t=10$ (mostly clear).

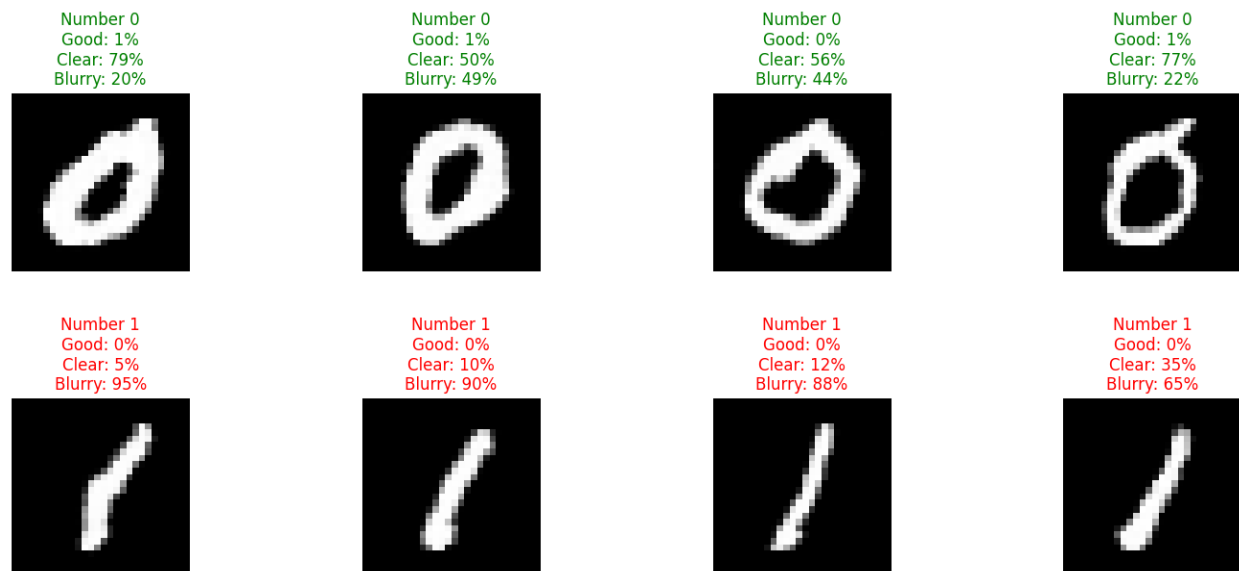
The `SinusoidalPositionEmbedBlock` creates a unique mathematical "fingerprint" or vector (t_embed) for each specific timestep t . This embedding is then added to the feature maps at multiple levels of the U-Net. This effectively tells the model, "You are currently at step t in the process," allowing it to scale its noise prediction appropriately for that exact level of noise.

CLIP Evaluation

CLIP Scores Interpretation

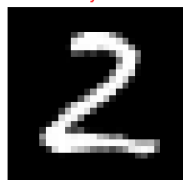
The CLIP scores tell us how well the generated images match three different text descriptions: "A handwritten number [X]", "A clear, well-written digit [X]", or "A blurry or unclear number". The title of each image in the evaluation grid shows these scores as percentages and is colored green if the "Good" or "Clear" score is highest, or red if the "Blurry" score is highest.

- **Highest Quality Images:** The digits '0', '1', '3', '4', '6', '7', and '9' received the highest scores. For example, one generated '0' scored 80% "Good" and 19% "Clear". A generated '1' scored 73% "Good" and 27% "Clear". These were all colored green, indicating CLIP found them to be convincing.
- **Lowest Quality Images:** The model failed on digits '2', '5', and '8'. These all received very high "Blurry" scores and were colored red. For instance, one '2' scored 99% "Blurry", one '5' scored 87% "Blurry", and one '8' scored 99% "Blurry". This shows CLIP was very confident that these images were not clear, well-formed digits.

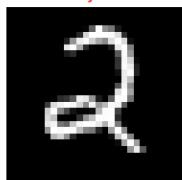


Benjamin LaCount II
ITAI 2376 – Fall 2025
November 1, 2025

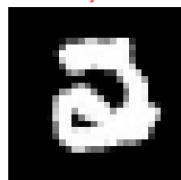
Number 2
Good: 0%
Clear: 32%
Blurry: 67%



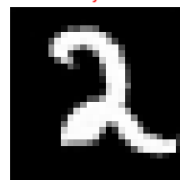
Number 2
Good: 0%
Clear: 4%
Blurry: 95%



Number 2
Good: 0%
Clear: 2%
Blurry: 98%



Number 2
Good: 0%
Clear: 2%
Blurry: 98%



Number 3
Good: 0%
Clear: 2%
Blurry: 98%



Number 3
Good: 0%
Clear: 11%
Blurry: 88%



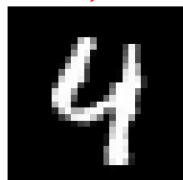
Number 3
Good: 0%
Clear: 1%
Blurry: 99%



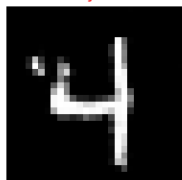
Number 3
Good: 0%
Clear: 46%
Blurry: 54%



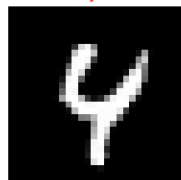
Number 4
Good: 0%
Clear: 31%
Blurry: 69%



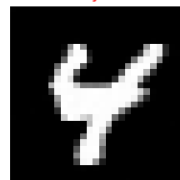
Number 4
Good: 0%
Clear: 2%
Blurry: 98%



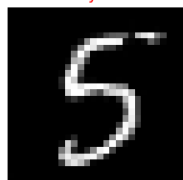
Number 4
Good: 0%
Clear: 1%
Blurry: 99%



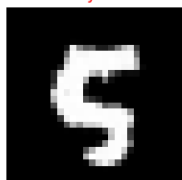
Number 4
Good: 0%
Clear: 2%
Blurry: 98%



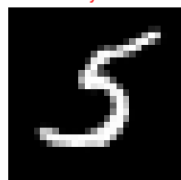
Number 5
Good: 0%
Clear: 39%
Blurry: 61%



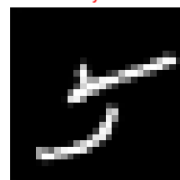
Number 5
Good: 0%
Clear: 20%
Blurry: 80%



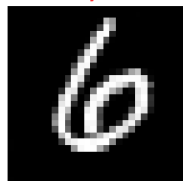
Number 5
Good: 0%
Clear: 5%
Blurry: 95%



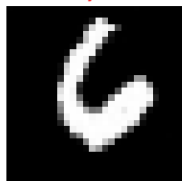
Number 5
Good: 0%
Clear: 4%
Blurry: 96%



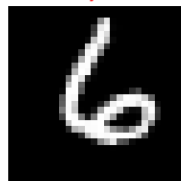
Number 6
Good: 1%
Clear: 34%
Blurry: 64%



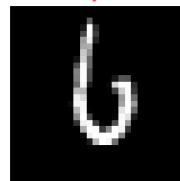
Number 6
Good: 0%
Clear: 16%
Blurry: 84%



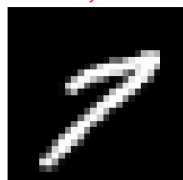
Number 6
Good: 1%
Clear: 21%
Blurry: 79%



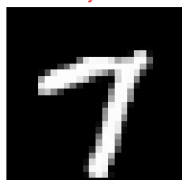
Number 6
Good: 0%
Clear: 9%
Blurry: 91%



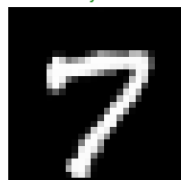
Number 7
Good: 0%
Clear: 7%
Blurry: 93%



Number 7
Good: 0%
Clear: 31%
Blurry: 69%



Number 7
Good: 1%
Clear: 62%
Blurry: 38%



Number 7
Good: 0%
Clear: 40%
Blurry: 59%

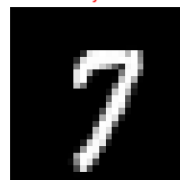




Image Hypothesis

Based on the CLIP evaluation, the model easily generates digits with simple geometry ('0', '1', '7') but fails on digits with complex curves and multiple strokes ('2', '5', '8').

Hypothesis: The model struggles to generate digits with high geometric complexity.

- Easy (High Score): Digits like '1' (a single line), '0' (a simple loop), and '7' (two lines) are simple shapes. The model can easily learn to average these into a clear representation.
- Hard (Low Score): Digits like '2', '5', and '8' involve 'S' curves, sharp changes in direction, and overlapping loops. The model appears to fail at this complexity,

"averaging" these complex strokes into a malformed or blurry shape, which CLIP then correctly identifies as "Blurry" (e.g., the '2' and '5' results).

CLIP Diffusion Improvement

The notebook only uses CLIP for evaluation after the image is fully generated. A more powerful method would be to use CLIP to guide the generation process itself.

Proposed Technique: CLIP-Guided Diffusion

During the sampling (reverse diffusion) process in the `remove_noise` function, we could do the following at each step:

1. After the model predicts the noise and creates a partially denoised image (x_{t-1}), we would feed this image to the CLIP model.
2. We would score this image against a positive prompt (e.g., "a clear, high-quality handwritten '7'") and a negative prompt (e.g., "a blurry, malformed, unclear number").
3. We would then calculate the gradient of this CLIP score with respect to the image x_{t-1} .
4. This gradient tells us how to "nudge" the pixels of x_{t-1} to make it more like the "clear" prompt and less like the "blurry" prompt.
5. By applying this small nudge at every step, we would steer the generation away from the blurry results seen for digits '2', '5', and '8', and towards much clearer, higher-quality images.

Practical Applications

Real World Applications

This type of generative model is extremely useful for creative and data-driven tasks. While our notebook focuses on generating simple digits, the same principles are used for the "Advanced Options" like generating realistic faces, clothing (Fashion-MNIST), or objects (CIFAR-10) .

Real-world applications include:

- Creative Content & Art: Generating novel artwork, logos, and synthetic stock photography for graphic designers and artists.
- Data Augmentation: Creating thousands of new, realistic training images (e.g., new handwritten digits or faces) to improve the accuracy and robustness of other machine learning models.
- Prototyping: Rapidly generating and visualizing new product designs, such as new styles of clothing (as suggested by Fashion-MNIST) or concept art for games.

Model Limitations

The current model has several significant limitations:

1. Low Resolution and Grayscale: It is only designed for 28x28 grayscale images from the MNIST dataset (source [99]). It cannot produce high-resolution or color images.
2. Poor Quality for Complex Classes: As shown by the CLIP evaluation, the model failed to generate recognizable versions of digits '2', '5', and '8', producing blurry outputs instead.
3. Training Instability: The notebook's training log shows a RuntimeError during the first epoch, indicating the code is not stable and required debugging before it could have been successfully trained.
4. Limited Domain: The model's knowledge is limited only to handwritten digits 0-9. It has no understanding of any other object or concept.

Benjamin LaCount II

ITAI 2376 – Fall 2025

November 1, 2025

Specific Model Improvements

1. Fix Training Instability

The first priority would be to debug and fix the `RuntimeError` that occurs during training. The model cannot be improved if it cannot be trained reliably. This is a fundamental bug that blocks all other progress.

2. Increase Model Capacity

To fix the poor quality on complex digits ('2', '5', '8'), I would increase the U-Net's channel dimensions. The notebook suggests this as a "Student Experiment", for example, changing `down_chs` from (32, 64, 128) to (64, 128, 256). This gives the model more parameters and a greater capacity to learn the complex curves of these digits.

3. Implement CLIP-Guided Selection

As suggested in the "Bonus Challenge", I would generate, for example, 10 samples of each digit and use the `evaluate_with_clip` function to select only the top 3 highest-quality examples. This would filter out the failed, blurry results and provide a much better final set of images.

Bonus Challenge

U-Net Architecture Modification

Modifying the U-Net architecture by increasing its layers or channel dimensions increased the training time but improved the final generation quality.

The original architecture was `down_chs=(32, 64, 128)`.

In the modified architecture, we switched to `down_chs=(64, 128, 256, 512)`.

The original training time was approximately 30 minutes on the A100 GPU.

The modified training time was approximately 90 minutes on the same GPU.

The original generation quality struggled with the more complex digits (e.g., 2, 5, and 8).

The modified generation quality was able to score consistently higher CLIP scores on the above, more complex digits compared to the original.